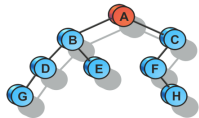
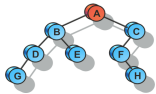


Алгоритмы программирования и структуры данных

Сбалансированные деревья поиска

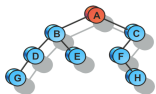


Splay-дерево: вставка и удаление



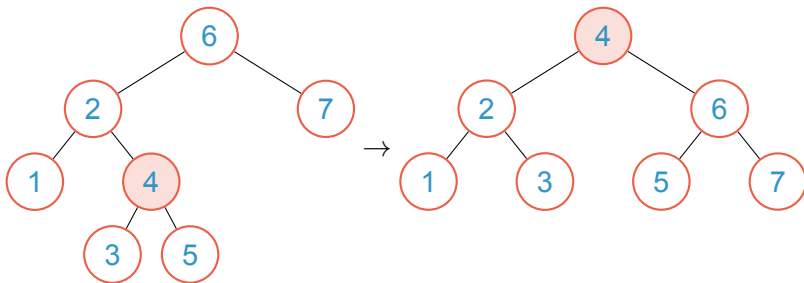
На прошлой лекции

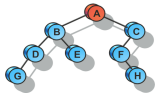
- **Splay-дерево** — самобалансирующееся двоичное дерево поиска без дополнительной информации в узлах с амортизированной сложностью основных операций, равной $O(\log n)$, достигающейся путем перемещения в корень узлов, с которыми производятся операции.



На прошлой лекции

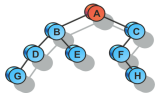
- ▶ **Splay-дерево** — самобалансирующееся двоичное дерево поиска без дополнительной информации в узлах с амортизированной сложностью основных операций, равной $O(\log n)$, достигающейся путем перемещения в корень узлов, с которыми производятся операции.
- ▶ Операция **Splay** за $O(\log n)$ перемещает узел в корень





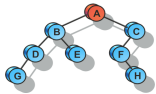
План лекции

- ▶ Поиск в Splay-дереве
 - ▶ С использованием операции Splay
- ▶ Операции разрезания и слияния
 - ▶ С использованием операций поиска и Splay
- ▶ Операции вставки и удаления
 - ▶ С использованием разрезания и слияния



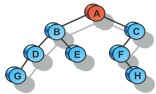
Поиск в Splay-дереве

- ▶ Проблема:
 - ▶ Если Splay-дерево вырождено (например, вытянуто вправо) и не изменяется при поиске элементов, то можно выполнить произвольное число операций поиска, при этом каждая будет выполняться за $O(n)$



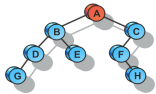
Поиск в Splay-дереве

- ▶ Проблема:
 - ▶ Если Splay-дерево вырождено (например, вытянуто вправо) и не изменяется при поиске элементов, то можно выполнить произвольное число операций поиска, при этом каждая будет выполняться за $O(n)$
- ▶ Решение:
 - ▶ При каждой операции поиска нужно выполнять операцию Splay



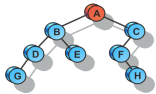
Поиск в Splay-дереве

- ▶ Проблема:
 - ▶ Если Splay-дерево вырождено (например, вытянуто вправо) и не изменяется при поиске элементов, то можно выполнить произвольное число операций поиска, при этом каждая будет выполняться за $O(n)$
- ▶ Решение:
 - ▶ При каждой операции поиска нужно выполнять операцию Splay
- ▶ Алгоритм поиска узла с ключом x :
 - ▶ Проведем поиск ключа x , как в обычном дереве поиска



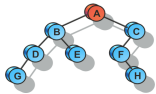
Поиск в Splay-дереве

- ▶ Проблема:
 - ▶ Если Splay-дерево вырождено (например, вытянуто вправо) и не изменяется при поиске элементов, то можно выполнить произвольное число операций поиска, при этом каждая будет выполняться за $O(n)$
- ▶ Решение:
 - ▶ При каждой операции поиска нужно выполнять операцию Splay
- ▶ Алгоритм поиска узла с ключом x :
 - ▶ Проведем поиск ключа x , как в обычном дереве поиска
 - ▶ Узел найден $\rightarrow$ выполним для него операцию Splay



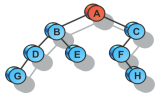
Поиск в Splay-дереве

- ▶ Проблема:
 - ▶ Если Splay-дерево вырождено (например, вытянуто вправо) и не изменяется при поиске элементов, то можно выполнить произвольное число операций поиска, при этом каждая будет выполняться за $O(n)$
- ▶ Решение:
 - ▶ При каждой операции поиска нужно выполнять операцию Splay
- ▶ Алгоритм поиска узла с ключом x :
 - ▶ Проведем поиск ключа x , как в обычном дереве поиска
 - ▶ Узел найден $\rightarrow$ выполним для него операцию Splay
 - ▶ Узел не найден $\rightarrow$ выполним операцию Splay для последнего просмотренного узла



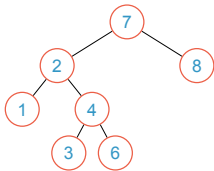
Поиск в Splay-дереве

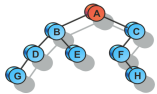
- ▶ Проблема:
 - ▶ Если Splay-дерево вырождено (например, вытянуто вправо) и не изменяется при поиске элементов, то можно выполнить произвольное число операций поиска, при этом каждая будет выполняться за $O(n)$
- ▶ Решение:
 - ▶ При каждой операции поиска нужно выполнять операцию Splay
- ▶ Алгоритм поиска узла с ключом x :
 - ▶ Проведем поиск ключа x , как в обычном дереве поиска
 - ▶ Узел найден $\rightarrow$ выполним для него операцию Splay
 - ▶ Узел не найден $\rightarrow$ выполним операцию Splay для последнего просмотренного узла
- ▶ Сложность: поиск в двоичном дереве + операция Splay $\rightarrow O(\log n)$



Примеры операции поиска

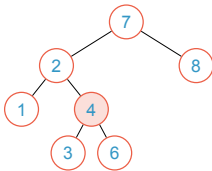
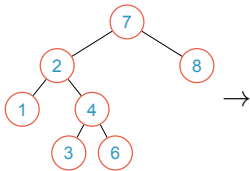
$x = 4$

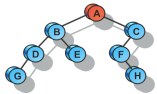




Примеры операции поиска

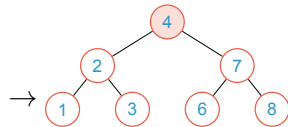
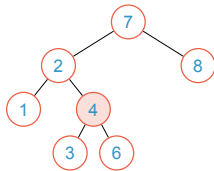
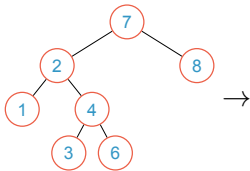
$x = 4$

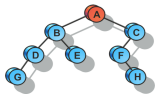




Примеры операции поиска

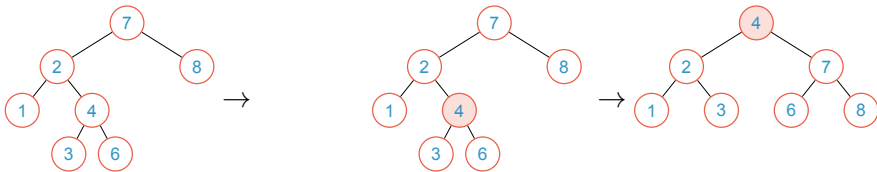
$x = 4$



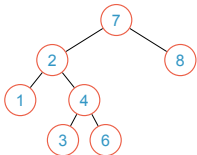


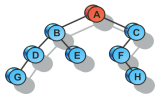
Примеры операции поиска

$x = 4$



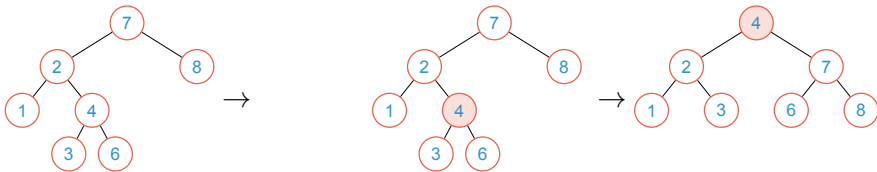
$x = 5$



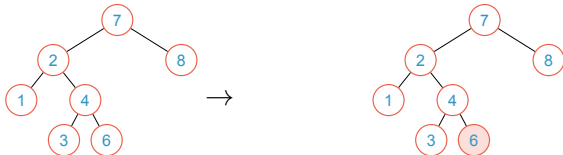


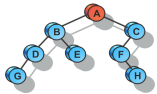
Примеры операции поиска

$x = 4$



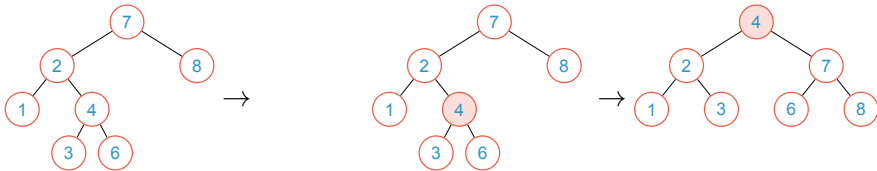
$x = 5$



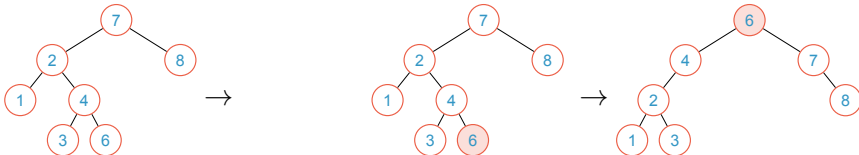


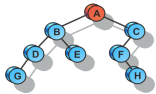
Примеры операции поиска

$x = 4$



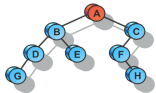
$x = 5$





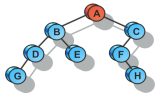
Операция разрезания

- ▶ Дано: splay-дерево T и ключ x
- ▶ Требуется: разделить T на два дерева L и R , чтобы все ключи в L были меньше x , а все ключи в R были не меньше x



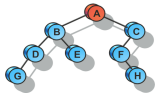
Операция разрезания

- ▶ Дано: splay-дерево T и ключ x
- ▶ Требуется: разделить T на два дерева L и R , чтобы все ключи в L были меньше x , а все ключи в R были не меньше x
- ▶ Алгоритм:
 1. T пустое $\rightarrow$ вернем два пустых дерева



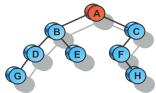
Операция разрезания

- ▶ Дано: splay-дерево T и ключ x
- ▶ Требуется: разделить T на два дерева L и R , чтобы все ключи в L были меньше x , а все ключи в R были не меньше x
- ▶ Алгоритм:
 1. T пустое $\rightarrow$ вернем два пустых дерева
 2. Выполним поиск ключа x



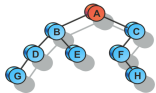
Операция разрезания

- ▶ Дано: splay-дерево T и ключ x
- ▶ Требуется: разделить T на два дерева L и R , чтобы все ключи в L были меньше x , а все ключи в R были не меньше x
- ▶ Алгоритм:
 1. T пустое $\rightarrow$ вернем два пустых дерева
 2. Выполним поиск ключа x
 - ▶ выполнится операция Splay



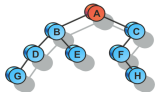
Операция разрезания

- ▶ Дано: splay-дерево T и ключ x
- ▶ Требуется: разделить T на два дерева L и R , чтобы все ключи в L были меньше x , а все ключи в R были не меньше x
- ▶ Алгоритм:
 1. T пустое $\rightarrow$ вернем два пустых дерева
 2. Выполним поиск ключа x
 - ▶ выполнится операция Splay
 3. Новый корень — узел y
 - ▶ l — левый ребенок y , r — правый



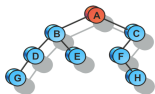
Операция разрезания

- ▶ Дано: splay-дерево T и ключ x
- ▶ Требуется: разделить T на два дерева L и R , чтобы все ключи в L были меньше x , а все ключи в R были не меньше x
- ▶ Алгоритм:
 1. T пустое $\rightarrow$ вернем два пустых дерева
 2. Выполним поиск ключа x
 - ▶ выполнится операция Splay
 3. Новый корень — узел y
 - ▶ l — левый ребенок y , r — правый
 4. Сравним ключ узла y с x :



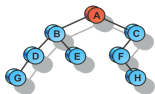
Операция разрезания

- ▶ Дано: splay-дерево T и ключ x
- ▶ Требуется: разделить T на два дерева L и R , чтобы все ключи в L были меньше x , а все ключи в R были не меньше x
- ▶ Алгоритм:
 1. T пустое $\rightarrow$ вернем два пустых дерева
 2. Выполним поиск ключа x
 - ▶ выполнится операция Splay
 3. Новый корень — узел y
 - ▶ l — левый ребенок y , r — правый
 4. Сравним ключ узла y с x :
 - ▶ больше: $L \leftarrow l, R \leftarrow \{y\} \cup r$



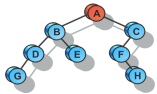
Операция разрезания

- ▶ Дано: splay-дерево T и ключ x
- ▶ Требуется: разделить T на два дерева L и R , чтобы все ключи в L были меньше x , а все ключи в R были не меньше x
- ▶ Алгоритм:
 1. T пустое $\rightarrow$ вернем два пустых дерева
 2. Выполним поиск ключа x
 - ▶ выполнится операция Splay
 3. Новый корень — узел y
 - ▶ l — левый ребенок y , r — правый
 4. Сравним ключ узла y с x :
 - ▶ больше: $L \leftarrow l, R \leftarrow \{y\} \cup r$
 - ▶ иначе: $L \leftarrow l \cup \{y\}, R \leftarrow r$



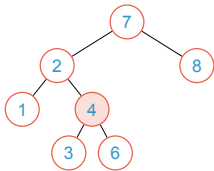
Операция разрезания

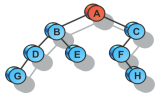
- ▶ Дано: splay-дерево T и ключ x
- ▶ Требуется: разделить T на два дерева L и R , чтобы все ключи в L были меньше x , а все ключи в R были не меньше x
- ▶ Алгоритм:
 1. T пустое $\rightarrow$ вернем два пустых дерева
 2. Выполним поиск ключа x
 - ▶ выполнится операция Splay
 3. Новый корень — узел y
 - ▶ l — левый ребенок y , r — правый
 4. Сравним ключ узла y с x :
 - ▶ больше: $L \leftarrow l, R \leftarrow \{y\} \cup r$
 - ▶ иначе: $L \leftarrow l \cup \{y\}, R \leftarrow r$
- ▶ Сложность: поиск + $O(1)$ операций $\rightarrow O(\log n)$



Примеры разрезания

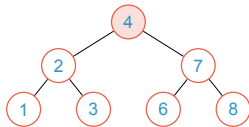
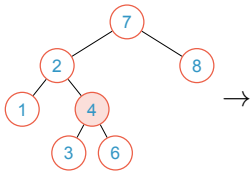
$x = 4$

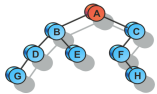




Примеры разрезания

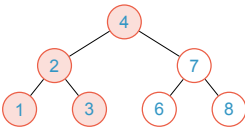
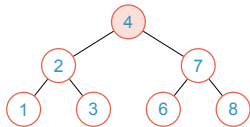
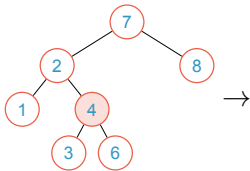
$x = 4$

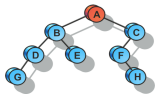




Примеры разрезания

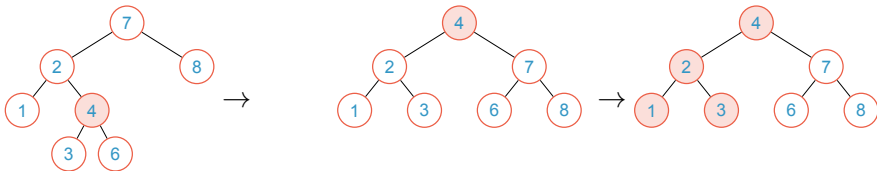
$x = 4$



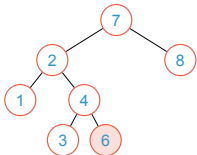


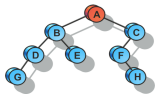
Примеры разрезания

$x = 4$



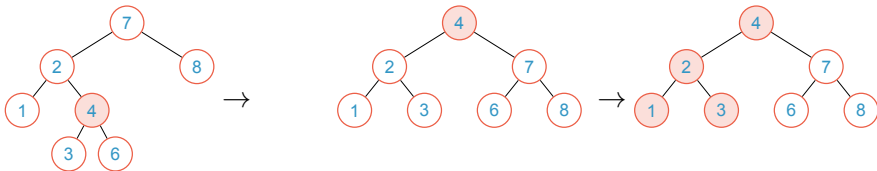
$x = 5$





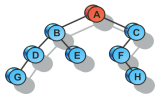
Примеры разрезания

$x = 4$



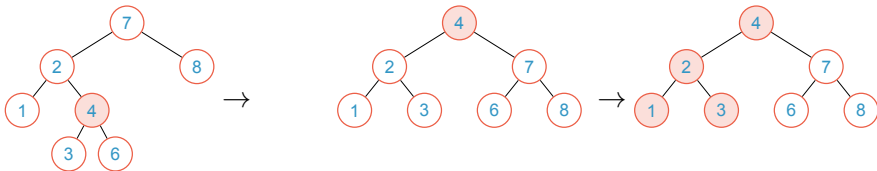
$x = 5$



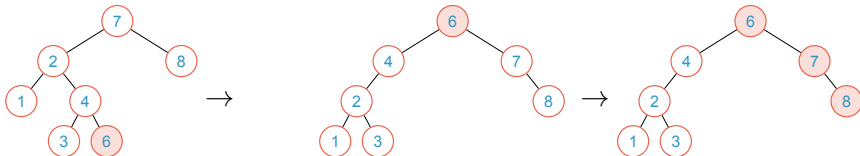


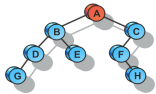
Примеры разрезания

$x = 4$



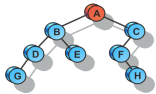
$x = 5$





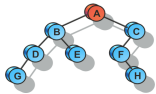
Операция слияния

- ▶ Операция, обратная операции разрезания
- ▶ Дано: два дерева L и R , каждый ключ из L меньше любого ключа из R
- ▶ Требуется: слить их в одно дерево



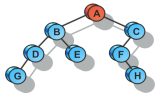
Операция слияния

- ▶ Операция, обратная операции разрезания
- ▶ Дано: два дерева L и R , каждый ключ из L меньше любого ключа из R
- ▶ Требуется: слить их в одно дерево
- ▶ Алгоритм:
 1. R пустое — вернуть L



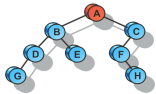
Операция слияния

- ▶ Операция, обратная операции разрезания
- ▶ Дано: два дерева L и R , каждый ключ из L меньше любого ключа из R
- ▶ Требуется: слить их в одно дерево
- ▶ Алгоритм:
 1. R пустое — вернуть L
 2. $y \leftarrow$ самый левый узел в R



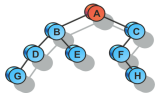
Операция слияния

- ▶ Операция, обратная операции разрезания
- ▶ Дано: два дерева L и R , каждый ключ из L меньше любого ключа из R
- ▶ Требуется: слить их в одно дерево
- ▶ Алгоритм:
 1. R пустое — вернуть L
 2. $y \leftarrow$ самый левый узел в R
 3. Выполним операцию Splay для y



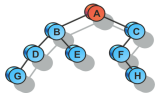
Операция слияния

- ▶ Операция, обратная операции разрезания
- ▶ Дано: два дерева L и R , каждый ключ из L меньше любого ключа из R
- ▶ Требуется: слить их в одно дерево
- ▶ Алгоритм:
 1. R пустое — вернуть L
 2. $y \leftarrow$ самый левый узел в R
 3. Выполним операцию Splay для y
 4. Левый ребенок y пустой. Сделаем L левым ребенком y

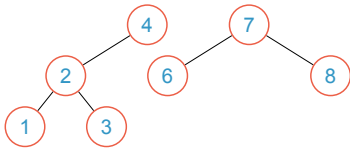


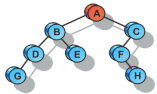
Операция слияния

- ▶ Операция, обратная операции разрезания
- ▶ Дано: два дерева L и R , каждый ключ из L меньше любого ключа из R
- ▶ Требуется: слить их в одно дерево
- ▶ Алгоритм:
 1. R пустое — вернуть L
 2. $y \leftarrow$ самый левый узел в R
 3. Выполним операцию Splay для y
 4. Левый ребенок y пустой. Сделаем L левым ребенком y
- ▶ Сложность: нахождение самого левого узла + операция Splay + $O(1)$ операций $\rightarrow O(\log n)$

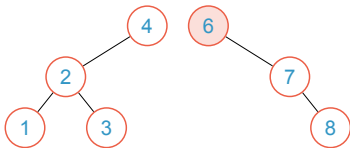
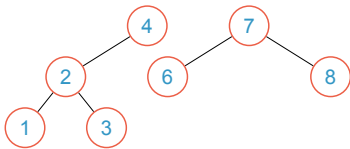


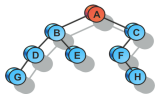
Пример слияния



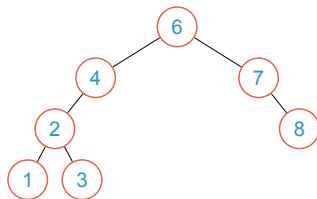
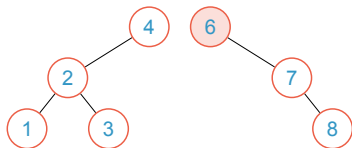
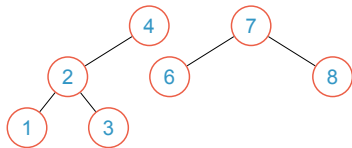


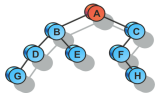
Пример слияния





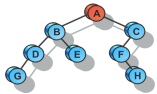
Пример слияния





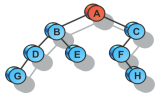
Операция вставки

- ▶ Алгоритм вставки ключа x :



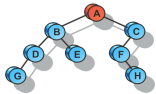
Операция вставки

- ▶ Алгоритм вставки ключа x :
 1. Осуществляем поиск x



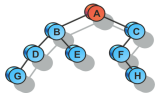
Операция вставки

- ▶ Алгоритм вставки ключа x :
 1. Осуществляем поиск x
 2. Если узел найден, ничего не делаем



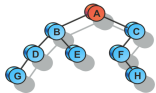
Операция вставки

- ▶ Алгоритм вставки ключа x :
 1. Осуществляем поиск x
 2. Если узел найден, ничего не делаем
 3. Иначе:



Операция вставки

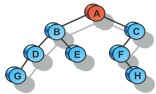
- ▶ Алгоритм вставки ключа x :
 1. Осуществляем поиск x
 2. Если узел найден, ничего не делаем
 3. Иначе:
 - ▶ Выполняем разрезание по ключу x (результат: деревья L и R)



Операция вставки

► Алгоритм вставки ключа x :

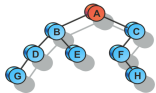
1. Осуществляем поиск x
2. Если узел найден, ничего не делаем
3. Иначе:
 - Выполняем разрезание по ключу x (результат: деревья L и R)
 - Создаем новый узел u с ключом x



Операция вставки

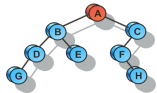
► Алгоритм вставки ключа x :

1. Осуществляем поиск x
2. Если узел найден, ничего не делаем
3. Иначе:
 - Выполняем разрезание по ключу x (результат: деревья L и R)
 - Создаем новый узел u с ключом x
 - Делаем L левым ребенком u , а R — правым ребенком u

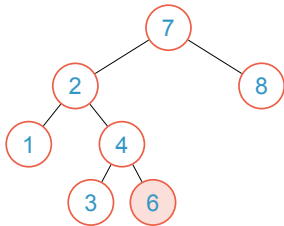


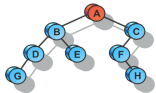
Операция вставки

- ▶ Алгоритм вставки ключа x :
 1. Осуществляем поиск x
 2. Если узел найден, ничего не делаем
 3. Иначе:
 - ▶ Выполняем разрезание по ключу x (результат: деревья L и R)
 - ▶ Создаем новый узел y с ключом x
 - ▶ Делаем L левым ребенком y , а R — правым ребенком y
- ▶ Сложность алгоритма: поиск + разрезание + $O(1)$ операций $\rightarrow O(\log n)$

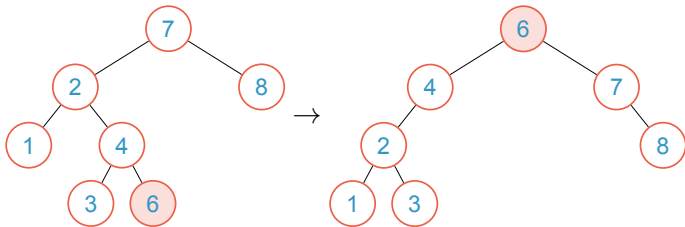


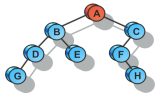
Пример вставки: $x = 5$



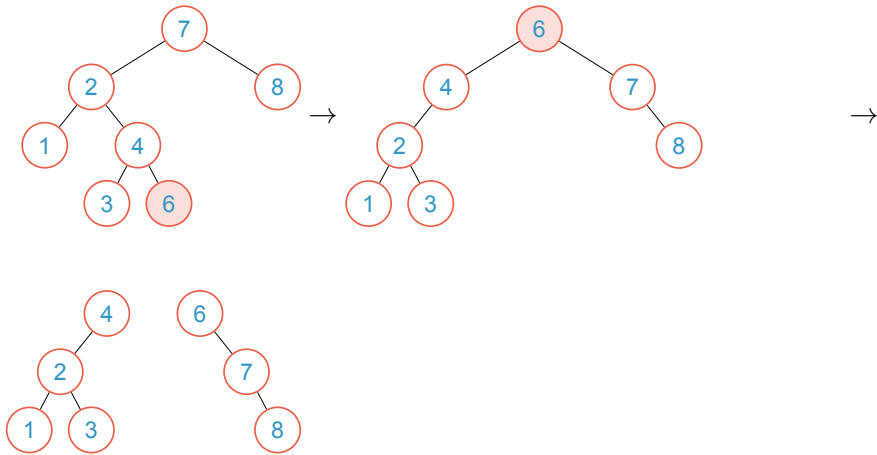


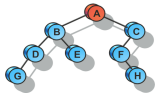
Пример вставки: $x = 5$



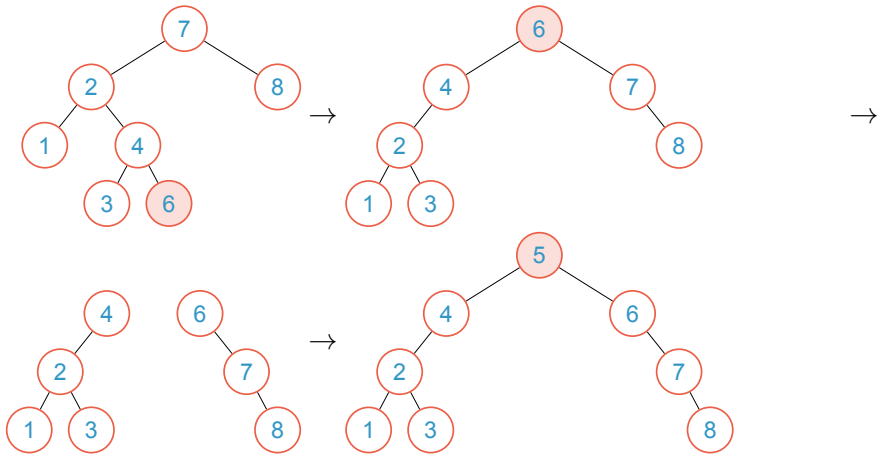


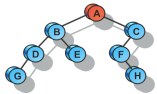
Пример вставки: $x = 5$





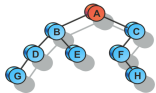
Пример вставки: $x = 5$





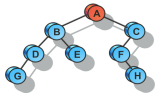
Операция удаления

- ▶ Алгоритм удаления узла с ключом x :



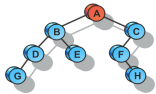
Операция удаления

- ▶ Алгоритм удаления узла с ключом x :
 1. Осуществляем поиск x



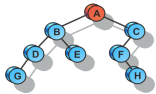
Операция удаления

- ▶ Алгоритм удаления узла с ключом x :
 1. Осуществляем поиск x
 2. Если узел не найден, ничего не делаем



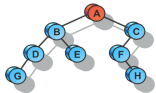
Операция удаления

- ▶ Алгоритм удаления узла с ключом x :
 1. Осуществляем поиск x
 2. Если узел не найден, ничего не делаем
 3. Иначе:



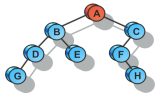
Операция удаления

- ▶ Алгоритм удаления узла с ключом x :
 1. Осуществляем поиск x
 2. Если узел не найден, ничего не делаем
 3. Иначе:
 - ▶ x является ключом корня



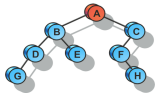
Операция удаления

- ▶ Алгоритм удаления узла с ключом x :
 1. Осуществляем поиск x
 2. Если узел не найден, ничего не делаем
 3. Иначе:
 - ▶ x является ключом корня
 - ▶ L — левый ребенок корня, R — правый ребенок корня



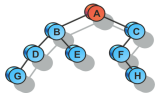
Операция удаления

- ▶ Алгоритм удаления узла с ключом x :
 1. Осуществляем поиск x
 2. Если узел не найден, ничего не делаем
 3. Иначе:
 - ▶ x является ключом корня
 - ▶ L — левый ребенок корня, R — правый ребенок корня
 - ▶ Удаляем корень



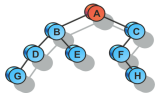
Операция удаления

- ▶ Алгоритм удаления узла с ключом x :
 1. Осуществляем поиск x
 2. Если узел не найден, ничего не делаем
 3. Иначе:
 - ▶ x является ключом корня
 - ▶ L — левый ребенок корня, R — правый ребенок корня
 - ▶ Удаляем корень
 - ▶ Выполняем слияние деревьев L и R

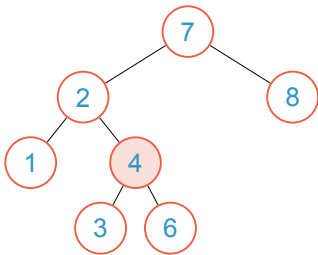


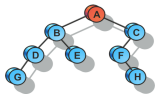
Операция удаления

- ▶ Алгоритм удаления узла с ключом x :
 1. Осуществляем поиск x
 2. Если узел не найден, ничего не делаем
 3. Иначе:
 - ▶ x является ключом корня
 - ▶ L — левый ребенок корня, R — правый ребенок корня
 - ▶ Удаляем корень
 - ▶ Выполняем слияние деревьев L и R
- ▶ Сложность алгоритма: поиск + слияние + $O(1)$ операций $\rightarrow O(\log n)$

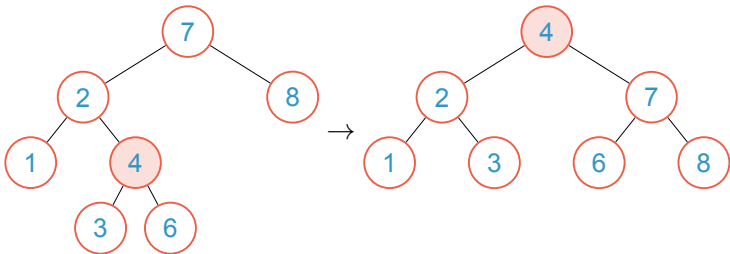


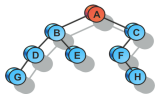
Пример удаления: $x = 4$



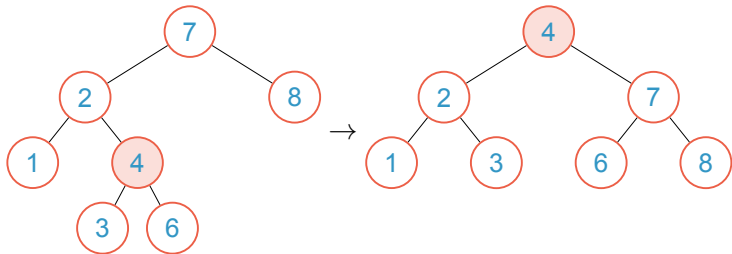


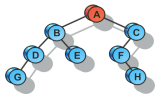
Пример удаления: $x = 4$



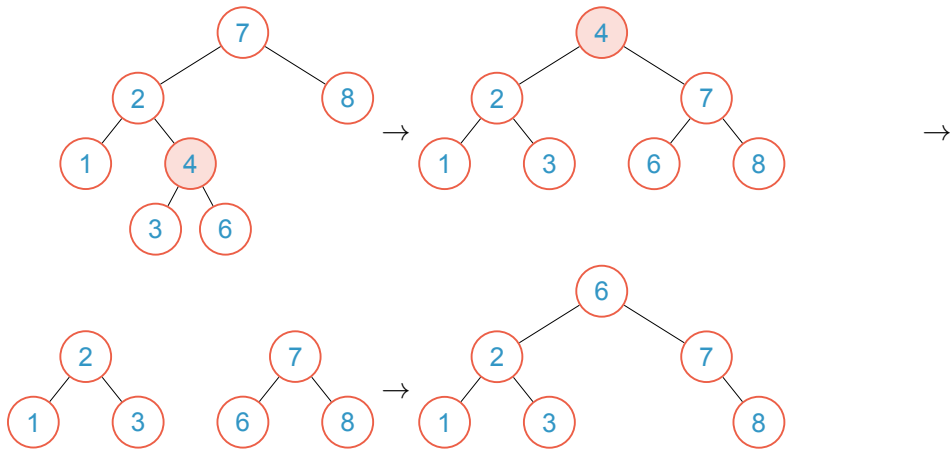


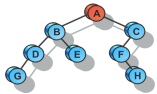
Пример удаления: $x = 4$





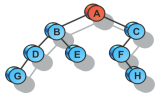
Пример удаления: $x = 4$





Заключение

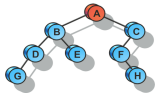
В этой лекции



Заключение

В этой лекции

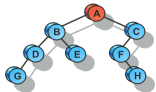
- Осуществление поиска в Splay-дереве



Заключение

В этой лекции

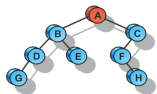
- ▶ Осуществление поиска в Splay-дереве
- ▶ Вставка и удаление в Splay-дереве



Заключение

В этой лекции

- ▶ Осуществление поиска в Splay-дереве
- ▶ Вставка и удаление в Splay-дереве
- ▶ Вспомогательные операции (используемые также и самостоятельно)
 - ▶ Разрезание и слияние

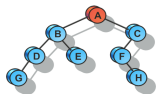


Заключение

В этой лекции

- ▶ Осуществление поиска в Splay-дереве
- ▶ Вставка и удаление в Splay-дереве
- ▶ Вспомогательные операции (используемые также и самостоятельно)
 - ▶ Разрезание и слияние

На следующей неделе



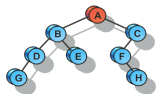
Заключение

В этой лекции

- ▶ Осуществление поиска в Splay-дереве
- ▶ Вставка и удаление в Splay-дереве
- ▶ Вспомогательные операции (используемые также и самостоятельно)
 - ▶ Разрезание и слияние

На следующей неделе

- ▶ Хэш-таблицы — эффективные структуры данных, основанные, в отличие от деревьев поиска, не на сравнении элементов



Заключение

В этой лекции

- ▶ Осуществление поиска в Splay-дереве
- ▶ Вставка и удаление в Splay-дереве
- ▶ Вспомогательные операции (используемые также и самостоятельно)
 - ▶ Разрезание и слияние

На следующей неделе

- ▶ Хэш-таблицы — эффективные структуры данных, основанные, в отличие от деревьев поиска, не на сравнении элементов

Спасибо за внимание!